About User Modules

User modules are self-contained logic programs that are represented as symbols in SIMPL. Programmers can bring a module into their SIMPL program and define its inputs, outputs, and parameters just as with any symbol.

Programmers create user modules using SIMPL, SIMPL+, or a combination of the two. That is, a module created with SIMPL may itself contain a SIMPL+ module. A module only contains logic symbols, not devices or any other hardware components.

The Crestron Modules folder of the Symbol Library contains hundreds of modules that have been programmed, tested and debugged at Crestron. You can use a Crestron module as the starting point for programming your own user module. Or you can create a new, blank user module.

SIMPL allows you to create the following user modules:

- User Serial Device Module: Generates serial strings in the format expected by a device such as a camera or audio/video switcher.
- User Logic Module: Adds custom logic that more easily accomplishes a system task. For example, you might create an Event Logger module that writes items to a file to track system usage. A user logic module does not connect to a device.

SIMPL also provides automation methods for creating the following connection logic modules:

- User Interface Connection Logic Module: Defines user interface programming and logic that can be imported into any Crestron touchpanel or keypad in a SIMPL program.
- User Crestron Device Connection Logic Module: Adds custom logic to drive a Crestron Ethernet or Cresnet device and to allow a SystemBuilder programmer to program the device inputs and outputs.
- User Front Panel Connection Logic Module: Creates a pre-defined front panel project that can be imported into a control system's front panel in SIMPL.

What you can do:

Create New User Modules...

Elements of a User Module

User Logic Module

User Serial Device Module

User Interface Connection Logic Module

User Crestron Device Connection Logic Module

User Front Panel Connection Logic Module

Program User Modules...

Define Signal Cues

Define Parameters

Set Signal Properties...

Digital Input Properties

Digital Output Properties

Analog Input/Output Properties

Serial Input/Output Properties

Parameter Properties

Edit Existing Modules...

Edit an Existing Module

SIMPL/SIMPL+ Modules Targeting 4-Series Control Systems

Each user created SIMPL/SIMPL+ module must target at least one control system type. Selecting a target implies that the module MUST work for that target and any statements that are not valid for that target are NOT permitted. Any attempt to add symbols or other logic that is not supported on the target control system(s), will be rejected.

Module programmers define targeted control system type(s) for SIMPL in the Module Header dialog; targeted control system type for SIMPL+ modules can be specified in the SIMPL+ toolbar or from the SIMPL+ **Build** menu.

Starting with SIMPL v. 4.14.xx the default target selection for a new SIMPL/SIMPL+ module is 3-Series AND 4-series control systems.

Note that on opening existing SIMPL+ modules SIMPL will add a 4-Series control system class target in addition to any previously defined control system class target(s). However, SIMPL will actually use the newly added 4-series target ONLY after the module is recompiled.

For example, if a SIMPL+ module was previously targeting 2-Series only, it will target 2-Series and 4-

Series control system after opening and saving that SIMPL+ module in SIMPL v. 4.14.xx and later.

SIMPL will add a 4-series control system target only to SIMPL modules previously targeting 3-series control systems. The 4-series selection will be used only after the SIMPL module is saved. Note that opening an existing SIMPL module targeting the 2-series control systems will not add the 4-series target the module.

	Module doesn't target 3 series in SIMPL v.4.11.06 or earlier	**Module targets 3-series (and/or other targets) in SIMPL v.4.11.06 or earlier**
SIMPL module	Module will target only existing targets	Module will target all other targets + 4-series control systems
SIMPL+ module	Module will target all other targets + 4-series control systems	Module will target all other targets + 4-series control systems

Adding symbols to SIMPL modules

Most symbols supported for 3-Series will also be supported for 4-Series by default.

Symbols that are not supported by the selected target control system(s) of the SIMPL module cannot be added to the module (via speed key or from the Symbol Library). If the targeted control system type is changed, symbols that are no longer supported will be commented out by SIMPL.

Adding SIMPL/SIMPL+ Module(s) to a SIMPL program

SIMPL/SIMPL+ modules can be added to the SIMPL program only if the modules target the control system present in the SIMPL program; a module cannot be added to the SIMPL program if that module does not target the control system present in the SIMPL program.

4-Series SIMPL/SIMPL+ modules in older version of SIMPL

4-Series SIMPL modules cannot be opened in older versions of SIMP: minimum required SIMPL version is 4.14.xx, minimum Device Database version is 115.00.001.

Most SIMPL+ modules targeting 4-Series could be opened in older versions of SIMPL, however the target control system selection will not be reflected in older SIMPL versions.

Note that SIMPL+ modules targeting 4-Series control systems and containing 4-Series CLZs, CANNOT be compiled in older SIMPL versions.

	In SIMPL v.4.11.06 or earlier
SIMPL module	Module cannot be opened if it targets 4-series
SIMPL+ module	Module targeting 4-series can be opened bit it won't compile if contains 4-series CLZ

Setting Target Control Systems for Existing Modules

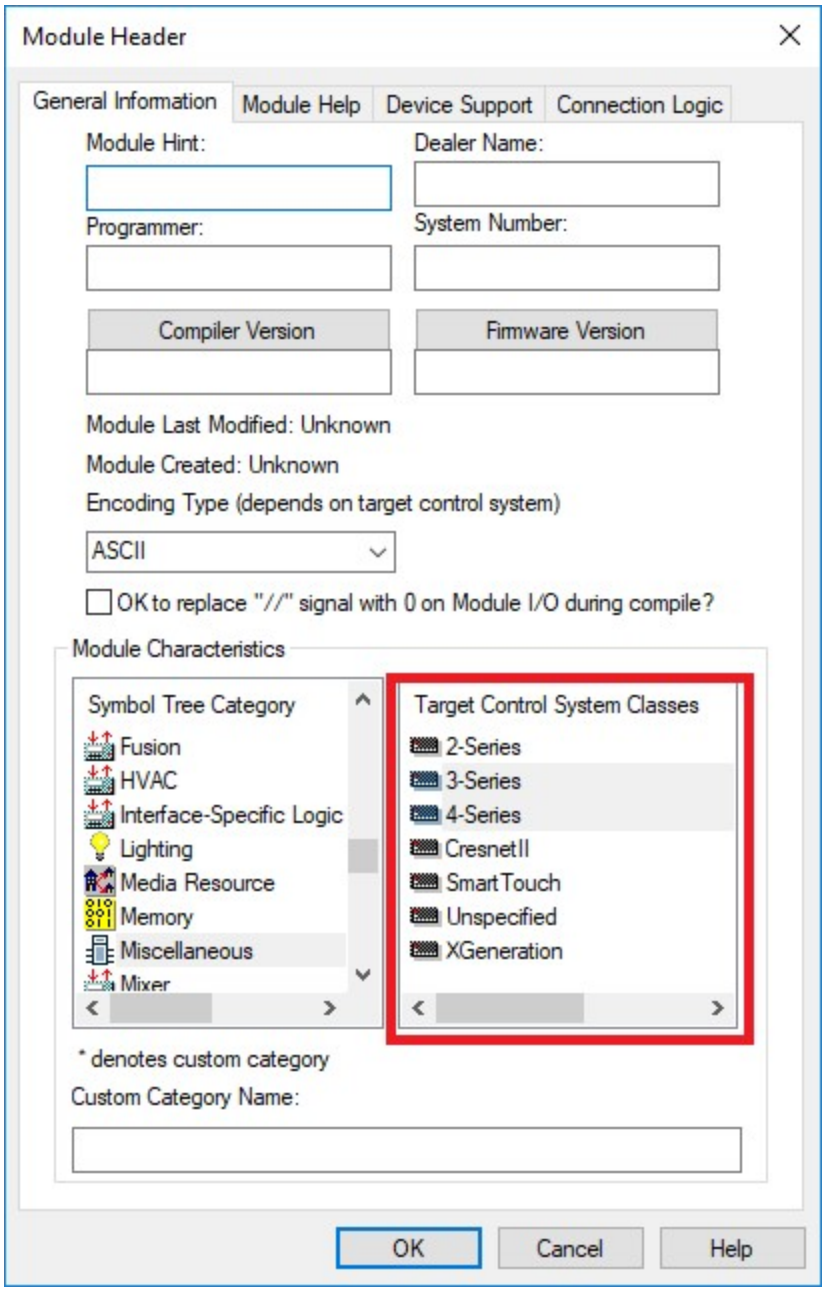
Existing modules targeted to work on 3-series control systems can be added seamlessly to a 4-series SIMPL program and will be valid for 4-series control systems, as well. No updates to the modules should be required.

> **NOTE:** On program open SIMPL 4.14.xx or later will display a compiler notice if your SIMPL program contains a SIMPL+ module utilizing the #IF_SERIES3 directive as all logic inside that directive will not be compiled into the 4-series program
>
> To resolve this issue, edit the SIMPL+ module and either remove the #IF_SERIES3 directive, or add the new #IF_SERIES4 directive and put the appropriate logic into that block.

Editing SIMPL modules created in older versions of SIMPL

Editing SIMPL modules created in older versions of SIMPL may require updates to the target control system selections.

After opening an older SIMPL module, make sure to open the Module Header dialog, review the target control systems selection and if necessary make changes as desired:



IMPORTANT: The new target control system selection will take place ONLY after the SIMPL module is saved.

Note that upon opening SIMPL module in the SIMPL Editor, any symbols which have been marked to be excluded from the current selected target control system classes, will be commented out and SIMPL Warning 153 will be added to the Error Log.

Elements of a User Module

Before you start programming a new user module, you should know exactly what its purpose and usage will be. You should verify that another module does not exist that already accomplishes the same task! And you should work out a general outline of the logic symbols and programming that the module will require.

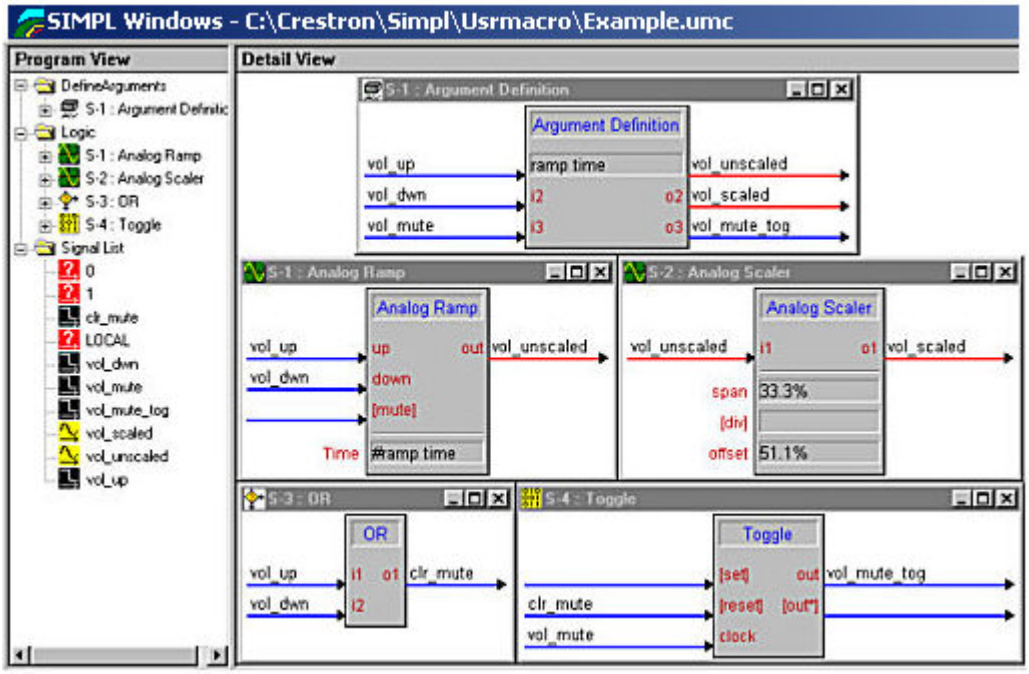
You should also have a good idea of what the Argument Definition will look like, including the following:

- The number and type of inputs, outputs and parameters.
- The default values, if any, for signals and parameters.
- The valid range of values for analog signals and parameters.
- The names of signal cues. These should be clear, unambiguous and easy for a user of the module to understand.

Elements of a user module

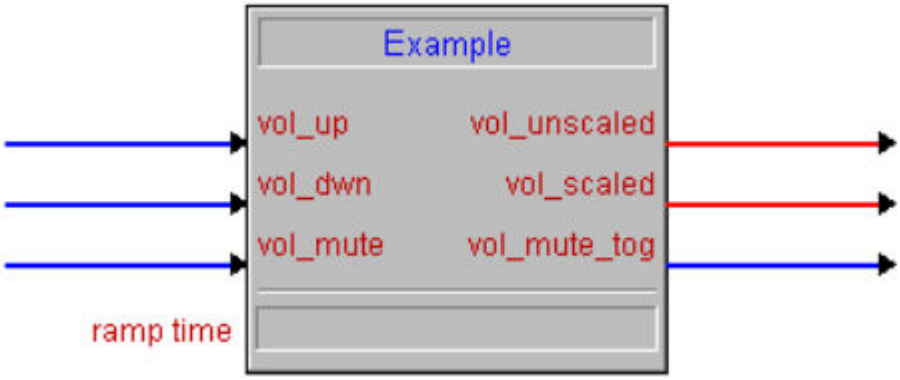
A module consists of an Argument Definition, which represents how the symbol will look in SIMPL, and logic symbols.

The inputs, outputs and parameters of the Argument Definition are associated with signals and parameters defined in the module.



The internal logic is not visible to users of the module. Thus the Argument Definition can be thought of as a black box; the user does not need to know the inner workings of the symbol to use it in a program.

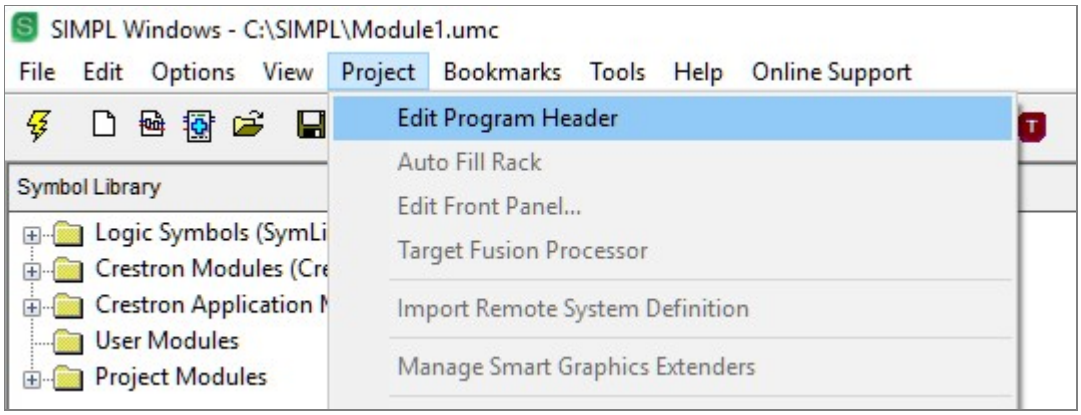
The signal and parameter names used in the Argument Definition become the signal cues that will be appear in the symbol detail.



When you compile a program containing a user module, the Argument Definition symbol disappears and all of the logic defined in the module is physically placed in the program.

Module Header

The **Module Header** dialog is displayed when a SIMPL module is first created. You can also edit the header of existing module by clicking **Edit Program Header** from the **Project** menu.



The **Module Header** dialog allows programmers to define general, module help, device support and connection logic properties of the new SIMPL module.

General Information Tab

Module Help Tab

Device Support Tab

Connection Logic Tab

User Serial Device Module

A user serial device module generates strings in the format required by a serial device such as a camera or digital audio processor.

Any user serial device module that is intended for use in SystemBuilder or D3 Pro must include a Virtual Serial Driver symbol, which connects the module to a COM port in the program. If the Virtual Serial Driver is not present in the module the serial device cannot be added to SystemBuilder or D3 Pro. Detailed instructions on creating serial device modules compatible with SystemBuilder and D3 Pro are provided in the help files for each program.

To create a user serial device module

Set up the Argument Definition

1. Click the **New SIMPL Module** button on the toolbar; or press Ctrl+M; or click **New SIMPL Module** on the **File** menu.

2. Edit the Module Header. (For user serial devices, do not select any options in the Connection Logic tab.)

3. Open the Argument Definition in Detail View and expand the symbol to add the required number of inputs, outputs and parameters.
    - Press Alt+Plus to add one signal/parameter field.
    - Press Alt+Shift+Plus to add multiple signals/parameter fields.
4. Define the signal cues.
5. Define the parameter fields.
6. Set optional properties, e.g., default values or status bar hints, for signals and parameters.
    - To set properties, select the signal/parameter and press Alt+F3, or right-click the signal and select **Edit Signal/Parameter Properties**.

Program the logic

1. Add the required logic symbols or modules to the Program View. Use subsystems to organize the programming just as in a standard SIMPL program.
2. Define the signals and parameters and connect them to their driving sources and destinations.
3. Connect the signals and parameters in the logic symbols to the associated inputs, outputs and parameters of the Argument Definition.
    - Any signals that will reference the signal cues in the Argument Definition must have the same signal name as the signal cue defined in the Argument Definition.
    - Input signals on logic symbols must be connected to the input side of the Argument Definition; outputs must be connected to the output side.
    - Parameters that reference the parameter fields in the Argument Definition must have the same parameter name, preceded by the pound (#) character. The parameter names are case sensitive—see Define Parameters.

      For example, if the Argument Definition has a parameter field called Ramp Time, then the logic symbol should have a parameter named #Ramp Time (no space between the pound character and the first letter of the parameter name).
    - Once you connect an input or output in the Argument Definition to signals in the module, the signal color will change from green (ambiguous) to the color representing the signal type: blue (digital), red (analog) or black (serial).

Program the Virtual Serial Driver symbol (SystemBuilder and D3 Pro modules only)

If the module is intended for use in SystemBuilder or D3 Pro, the module must include a Virtual Serial Driver symbol, which connects the module to a COM port in the program.

The Virtual Serial Driver is connected to the Argument Definition as follows:

1. Connect the <**tx$**> input of the Virtual Serial driver to the serial *output* of the Argument Definition.

2. Connect the <**rx$**> output of the Virtual Serial driver to the serial *input* of the Argument Definition.

3. Connect any optional signals of the Virtual Serial Driver to the Argument Definition, if they have been defined. The inputs of the Virtual Serial Driver connect to the *outputs* of the Argument Definition; the outputs of the Virtual Serial Driver connect to the *inputs* of the Argument Definition.

4. Use the parameters of the Virtual Serial Driver to specify the communication settings of the device.

Save the module

1. Click the **Save** button, or click **Save** on the **File** menu.

2. Save the module in the designated User Module directory.

User Logic Module

A User Logic Module adds custom logic that more easily accomplishes a system task. For example, you might create an Event Logger module that writes items to a file to track system usage. A user logic module does not connect to a device.

To create a user logic module

Set up the Argument Definition

1. Click the **New SIMPL Module** button on the toolbar; or press Ctrl+M; or click **New SIMPL Module** on the **File** menu.

2. Edit the Module Header. (For a user logic module do not select any options in the Connection Logic tab of the module header.)

   If the logic module is intended for use in SystemBuilder, click the Device Support tab of the module header and select "(Logic)" from the Device Family list. This will make the module visible in the User Logic Modules library in the SystemBuilder Programming View. If "(Logic)" is not selected the module will not be visible in SystemBuilder.

3. Open the Argument Definition in Detail View and expand the symbol to add the required number of inputs, outputs and parameters.

   - Press Alt+Plus to add one signal/parameter field.

- Press Alt+Shift+Plus to add multiple signals/parameter fields.

4. Define the signal cues.

5. Define the parameter fields.

6. Set optional properties, e.g., default values or status bar hints, for signals and parameters.

   - To set properties, select the signal/parameter and press Alt+F3, or right-click the signal and select **Edit Signal/Parameter Properties**.

Program the logic

1. Add the required logic symbols or modules to the Program View. Use subsystems to organize the programming just as in a standard SIMPL program.

2. Define the signals and parameters and connect them to their driving sources and destinations.

3. Connect the signals and parameters in the logic symbols to the associated inputs, outputs and parameters of the Argument Definition.

   - Any signals that will reference the signal cues in the Argument Definition must have the same signal name as the signal cue defined in the Argument Definition.

   - Input signals on logic symbols must be connected to the input side of the Argument Definition; outputs must be connected to the output side.

   - Parameters that reference the parameter fields in the Argument Definition must have the same parameter name, preceded by the pound (#) character. The parameter names are case sensitive—see Define Parameters.

     For example, if the Argument Definition has a parameter field called Ramp Time, then the logic symbol should have a parameter named #Ramp Time (no space between the pound character and the first letter of the parameter name).

   - Once you connect an input or output in the Argument Definition to signals in the module, the signal color will change from green (ambiguous) to the color representing the signal type: blue (digital), red (analog) or black (serial).

Save the module

1. Click the **Save** button, or click **Save** on the **File** menu.

2. Save the module in the designated User Module directory.

User TCP/IP Devices

A user TCP/IP device connection logic module adds custom logic to communicate with a third-party Ethernet device and to allow a SystemBuilder programmer to program the device inputs and outputs.

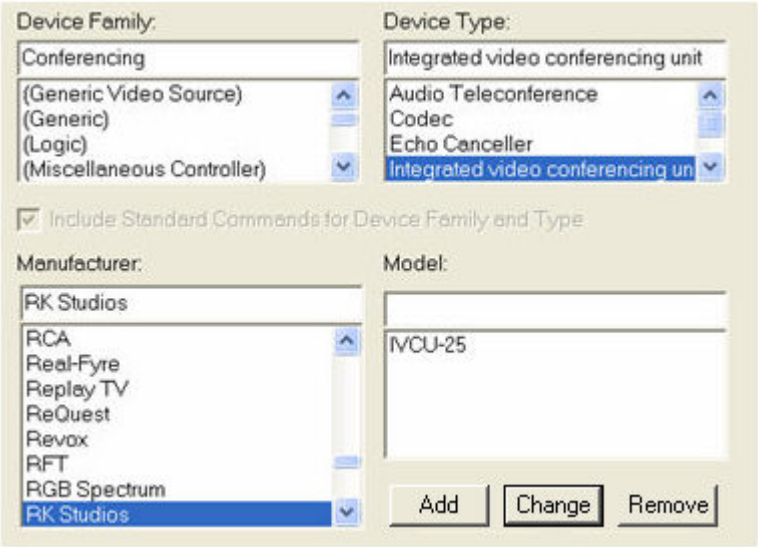
To create a user TCP/IP device connection logic module

Configure the module header and add the Virtual Device for Module Connection Symbol

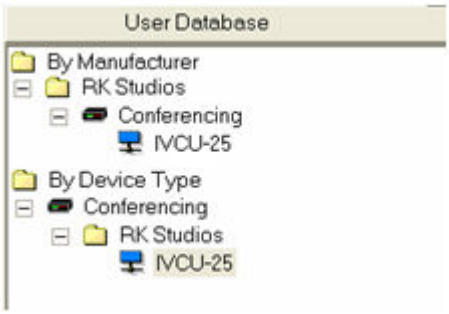
1. Click the **New SIMPL Module** button on the toolbar; or press Ctrl+M; or click **New SIMPL Module** on the **File** menu.

2. On the Module Header, click the Device Support tab. This tab contains options that determine how the device will be organized within the User Module library in SystemBuilder. You can select items from the Device Family, Device Type, Manufacturer and Model lists, or type custom names in the text fields above the lists.

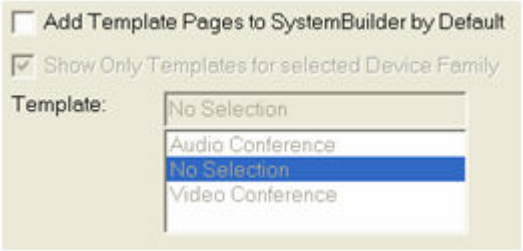
   For example, given the following selections:



   SystemBuilder will assign the Device Type as the name of the device, and will organize the device as follows in the SystemBuilder User Module library:

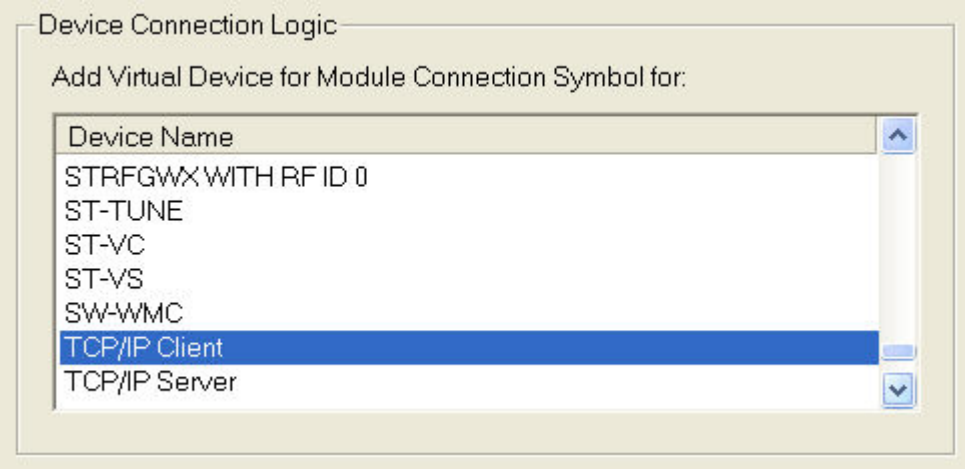


3. Clear (de-select) the **Add Template Pages** check box, since the third-party TCP/IP device will require custom templates.



4. Click the Connection Logic tab.

5. In the Device Connection Logic area, select the **TCP/IP Client** device from the Device list to add a Virtual Device for Module Connection symbol, which will contain the TCP/IP Client logic and will automatically be connected to the Argument Definition.



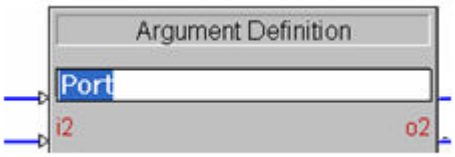
6. Click **OK** to close the module header.

7. Upon closing the module header, a prompt will be displayed that allows you to specify an optional prefix that will added to all signals being used in the default logic. Type the prefix in the text field, or leave the text field blank (for no prefix) and click **OK**.

Add and configure the Port parameter, which specifies the port connection information

1. Open the Argument Definition and double-click the first available parameter field.

2. Type "Port" (with no parentheses) in the parameter field.

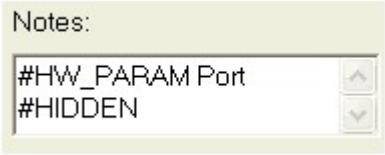


3. Right-click the Port parameter and select **Edit Parameter Properties**.

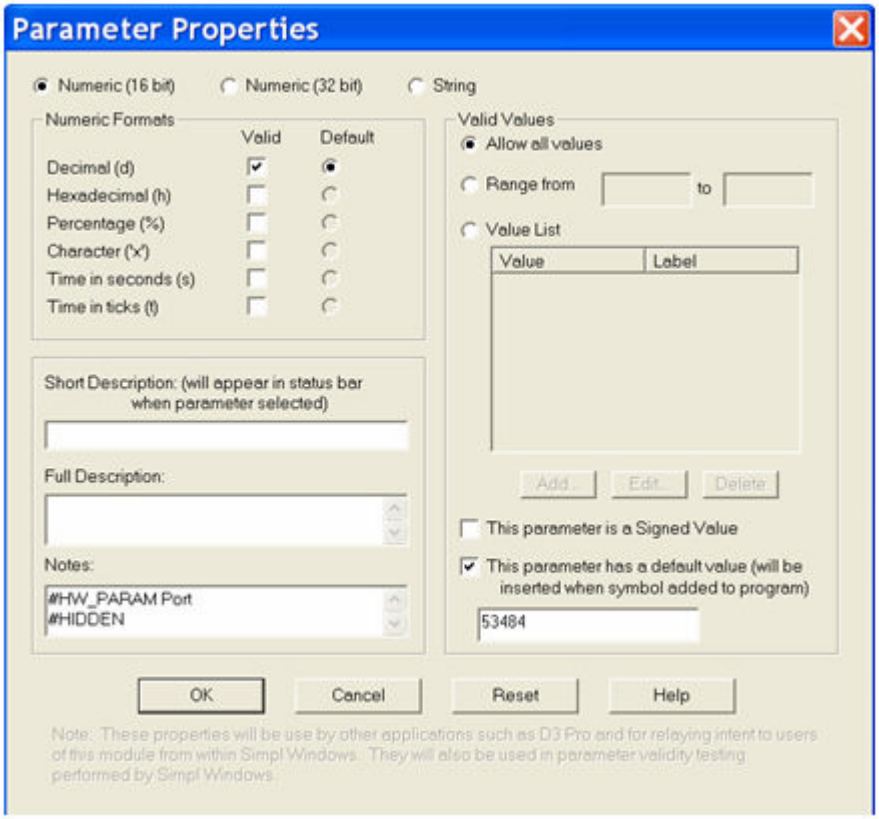
4. Edit the parameter properties as follows, to link the parameter in the Argument Definition to the Port parameter of the TCP/IP Client symbol:

- Click the **Numeric (16 bit)** radio button to select the 16-bit numeric format.
- Select the **Decimal (d)** numeric format check box, and clear (de-select) all the other numeric formats.
- In the Notes field, type the following: **#HW_PARAM Port**. This is an instruction to SystemBuilder to retrieve the Port number from the Port parameter. The note must be in the correct format, starting with the pound # character followed by HW_PARAM (all caps with the underscore and no spaces), followed by a space, followed by the name of the parameter (which in this case is Port).

  To hide the parameter properties from SystemBuilder so that the SystemBuilder programmer cannot edit the port number, type the directive **#HIDDEN** in a new line.



- Click the **Allow all values** radio button.
- Click the check box for **This parameter has a default value** to enable the default value text field. Type the Port number for communication with the TCP/IP device in the text field. The Port number should be obtained from the manufacturer of the device.



5. Click **OK** to close the Parameter Properties.

Program and save the module

1. Add any additional SIMPL or SIMPL+ logic to the module as required. You can change any signal names, as long you maintain the connection between the symbols and whatever logic you are using to generate/parse the data. That is, you must change the name everywhere it appears. Always name the signals using standard conventions, such as those used on Crestron modules and IR drivers. (See below for additional information on signal formatting.)

2. Verify that all inputs and outputs to additional logic symbols are also connected to the module's Argument Definition.

3. Save the file in the designated User Module directory.

Upon saving the module in the User Module directory, the new TCP/IP device can now be added in the Equipment View, and its inputs and outputs programmed in the Programming View.

Signal Formatting and Automatic Programming

SIMPL uses the following naming convention when automatically generating connection logic:

- Virtual Device symbol inputs: {{Prefix_Device_Cue_>>Device_Name}}

- Virtual Device symbol outputs: {{Prefix_Device_Name>>_Device_Cue}}

The double curly braces {{ }} around the signal name signify that the signal will be hidden from SystemBuilder.

The Virtual Device symbol is automatically connected to the Argument Definition as follows:

- The inputs to the Virtual Device symbol are connected to the *outputs* of the Argument Definition.

  Therefore, the format of the outputs on the Argument Definition is:
  {{Prefix_Device_Cue_>>Device_Name}}

- The outputs of the Virtual Device symbol are connected to the *inputs* of the Argument Definition.

  Therefore, the format of the inputs to the Argument Definition is:
  {{Prefix_Device_Name>>_Device_Cue}}

To make the device programmable in SystemBuilder, SIMPL provides the following additional connections:

- For every signal cue that defines an input or output in the Virtual Device symbol (via the <**cue #**> parameters), a corresponding signal is added to the Argument Definition without the double curly braces, signifying that the signal will be visible and programmable in SystemBuilder.

- Every digital, analog and serial signal that is to be visible in SystemBuilder is connected to a Digital Force, Analog Force, and Serial Force symbol, respectively.

- All digital signals are connected to Buffer symbols (with the <**enable**> line set to 1).
- All analog and serial signals are connected to Serial Buffer symbols (with the <**enable**> line set to 1).

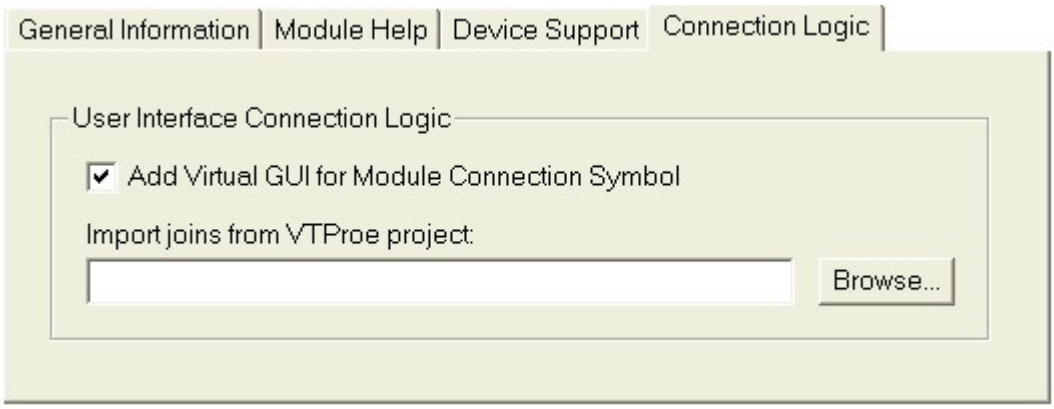
User Interface Connection Logic Module

A user interface connection logic module defines the programming for an interface such as a touchpanel or keypad, which can then be imported into a SIMPL program to quickly program an interface or connect it to a device.

To create a user interface connection logic module

Add the Virtual GUI for Module Connection Symbol

1. Click the **New SIMPL Module** button on the toolbar; or press Ctrl+M; or click **New SIMPL Module** on the **File** menu.
2. On the Module Header, click the Connection Logic tab.
3. Select the **Add Virtual GUI for Module Connection** check box. This will add a Virtual GUI symbol to the Logic subsystem.



4. To import joins from an existing VT Pro-e project, click **Browse** and locate the .vtp file. When you have located the file click **Open**. This will temporarily open VT Pro-e and cycle through all the pages in the project as joins are imported into the module. The imported joins will automatically be defined on the Virtual GUI symbol and connected to the Argument Definition.

   SIMPL does not import every join type that can be assigned in VT Pro-e. Any un-imported joins should be programmed manually.

5. Click **OK** to close the module header.
6. Open the Virtual GUI symbol in Detail View and program the symbol the same as a touchpanel or keypad.

Set up the Argument Definition

1. Connect the Virtual GUI symbol to the Argument Definition as follows:
    - Connect the outputs of the Virtual GUI to the *inputs* of the Argument Definition.
    - Connect the inputs of the Virtual GUI to the *outputs* of the Argument Definition.
2. Open the Argument Definition in Detail View and expand the symbol to add the required number of inputs, outputs and parameters.
    - Press Alt+Plus to add one signal/parameter field.
    - Press Alt+Shift+Plus to add multiple signals/parameter fields.
3. Set optional properties, e.g., default values or status bar hints, for signals and parameters.
    - To set properties, select the signal/parameter and press Alt+F3, or right-click the signal and select **Edit Signal/Parameter Properties**.

Program the logic

1. Add the required logic symbols or modules to the Program View. Use subsystems to organize the programming just as in a standard SIMPL program.
2. Define the signals and parameters and connect them to their driving sources and destinations.
3. Connect the signals and parameters in the logic symbols to the associated inputs, outputs and parameters of the Argument Definition.
    - Any signals that will reference the signal cues in the Argument Definition must have the same signal name as the signal cue defined in the Argument Definition.
    - Input signals on logic symbols must be connected to the input side of the Argument Definition; outputs must be connected to the output side.
    - Parameters that reference the parameter fields in the Argument Definition must have the same parameter name, preceded by the pound (#) character. The parameter names are case sensitive—see Define Parameters.

      For example, if the Argument Definition has a parameter field called Ramp Time, then the logic symbol should have a parameter named #Ramp Time (no space between the pound character and the first letter of the parameter name).
    - Once you connect an input or output in the Argument Definition to signals in the module, the signal color will change from green (ambiguous) to the color representing the signal type: blue (digital), red (analog) or black (serial).

Save the module

1. Click the **Save** button, or click **Save** on the **File** menu.

2. Save the module in the designated User Module directory.

User Crestron Device Connection Logic Module

A user Crestron device connection logic module adds custom logic to drive a Crestron Ethernet or Cresnet device and to allow a SystemBuilder programmer to program the device inputs and outputs.

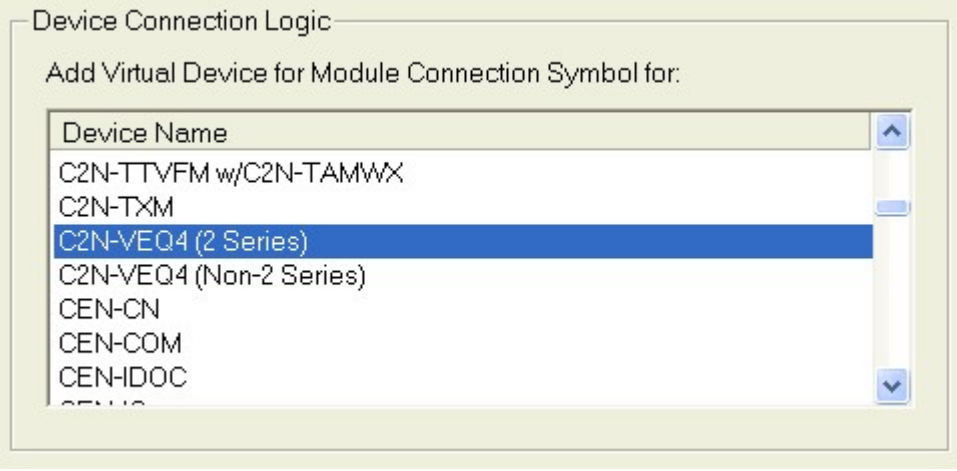
To create a user Crestron device connection logic module

Add the Virtual Device for Module Connection Symbol

1. Click the **New SIMPL Module** button on the toolbar; or press Ctrl+M; or click **New SIMPL Module** on the **File** menu.

2. On the Module Header, click the Connection Logic tab.

3. Select the Crestron device from the Device Name list to add a Virtual Device for Module Connection symbol. This will add the default logic that allows the module to drive the Crestron device. If the device is sub-slotted, a separate Virtual Device symbol will be added for each sub-slot. Each Virtual Device symbol will be automatically connected to the Argument Definition.



4. Click **OK** to close the module header.

5. Upon closing the module header, a prompt will be displayed that allows you to specify an optional prefix that will added to all signals being used in the default logic. Type the prefix in the text field, or leave the text field blank (for no prefix) and click **OK**.

Signal Formatting and Automatic Programming

SIMPL uses the following naming convention when automatically generating connection logic:

- Virtual Device symbol inputs: {{Prefix_Device_Cue_>>_Device_Name}}
- Virtual Device symbol outputs: {{Prefix_Device_Name_>>_Device_Cue}}

The double curly braces {{ }} around the signal name signify that the signal will be hidden from the SystemBuilder interface.

The Virtual Device symbol is automatically connected to the Argument Definition as follows:

- The inputs to the Virtual Device symbol are connected to the *outputs* of the Argument Definition.

  Therefore, the format of the outputs on the Argument Definition is:
  {{Prefix_Device_Cue_>>_Device_Name}}

- The outputs of the Virtual Device symbol are connected to the *inputs* of the Argument Definition.

  Therefore, the format of the inputs to the Argument Definition is:
  {{Prefix_Device_Name_>>_Device_Cue}}

To make the device programmable in SystemBuilder, SIMPL provides the following additional connections:

- For every signal cue that defines an input or output in the Virtual Device symbol (via the <**cue #**> parameters), a corresponding signal is added to the Argument Definition without the double curly braces, signifying that the signal will be visible and programmable in SystemBuilder.
- Every digital, analog and serial signal that is to be visible in SystemBuilder is connected to a Digital Force, Analog Force, and Serial Force symbol, respectively.
- All digital signals are connected to Buffer symbols (with the <**enable**> line set to 1).
- All analog and serial signals are connected to Serial Buffer symbols (with the <**enable**> line set to 1).

Program additional logic if required

1. Add any additional logic symbols or modules to the Program View. Use subsystems to organize the programming just as in a standard SIMPL program.
2. Define the signals and parameters and connect them to their driving sources and destinations.
3. Connect the signals and parameters in the logic symbols to the associated inputs, outputs and parameters of the Argument Definition.
   - Any signals that will reference the signal cues in the Argument Definition must have the same signal name as the signal cue defined in the Argument Definition.
   - Input signals on logic symbols must be connected to the input side of the Argument Definition; outputs must be connected to the output side.

- Parameters that reference the parameter fields in the Argument Definition must have the same parameter name, preceded by the pound (#) character. The parameter names are case sensitive—see Define Parameters.

  For example, if the Argument Definition has a parameter field called Ramp Time, then the logic symbol should have a parameter named #Ramp Time (no space between the pound character and the first letter of the parameter name).

- Once you connect an input or output in the Argument Definition to signals in the module, the signal color will change from green (ambiguous) to the color representing the signal type: blue (digital), red (analog) or black (serial).

Save the module

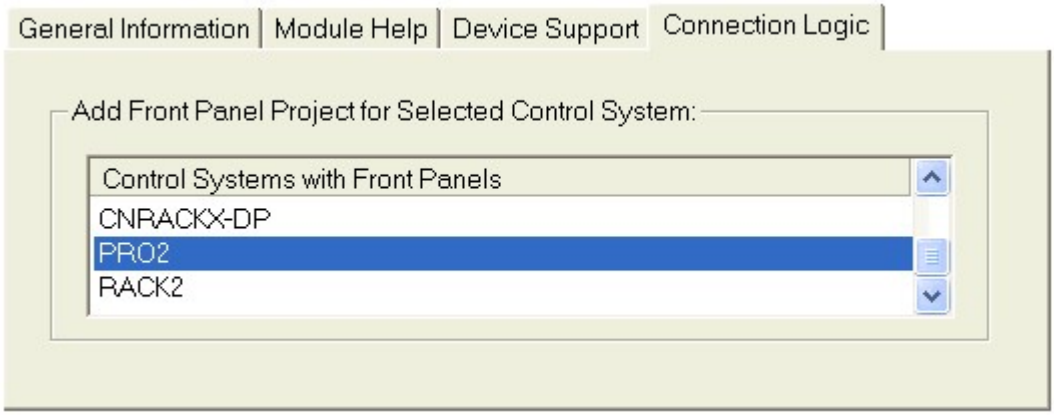
1. Click the **Save** button, or click **Save** on the **File** menu.
2. Save the module in the designated User Module directory.

User Front Panel Connection Logic Module

A user front panel connection logic module creates a pre-defined front panel project that can be imported into a SIMPL program to quickly program a front panel.

To create a user front panel connection logic module

1. Click the **New SIMPL Module** button on the toolbar; or press Ctrl+M; or click **New SIMPL Module** on the **File** menu.
2. On the Module Header, click the Connection Logic tab.
3. Select the control system from the "Control Systems with Front Panels" list.



4. Click **OK** to close the module header.

5. Click **Edit Front Panel** on the **Project** menu. This will open the Front Panel Editor.

6. Edit the front panel project.

The user front panel connection logic module does not require any programming of the Argument Definition.

Save the module

1. Click the **Save** button, or click **Save** on the **File** menu.

2. Save the module in the designated User Module directory.

Define Signal Cues

To name the inputs and outputs of the Argument Definition c lick each signal to highlight it and type the name in the signal field.

The signal NAMES you type become the signal CUES of the module's symbol.

- For clarity it is best to arrange the signals by type: digitals, analogs, and serials.

- Signal cues can consist of letters, numbers or characters. However, signal cues cannot begin with a number.

- The comma character (,) and colon character (:) are not valid.

You cannot have any gaps between cues. That is, you must define continuous signals starting with i1/o1. If you leave a signal position blank, any signal cues defined after the blank signal will be deleted and an error will be generated.

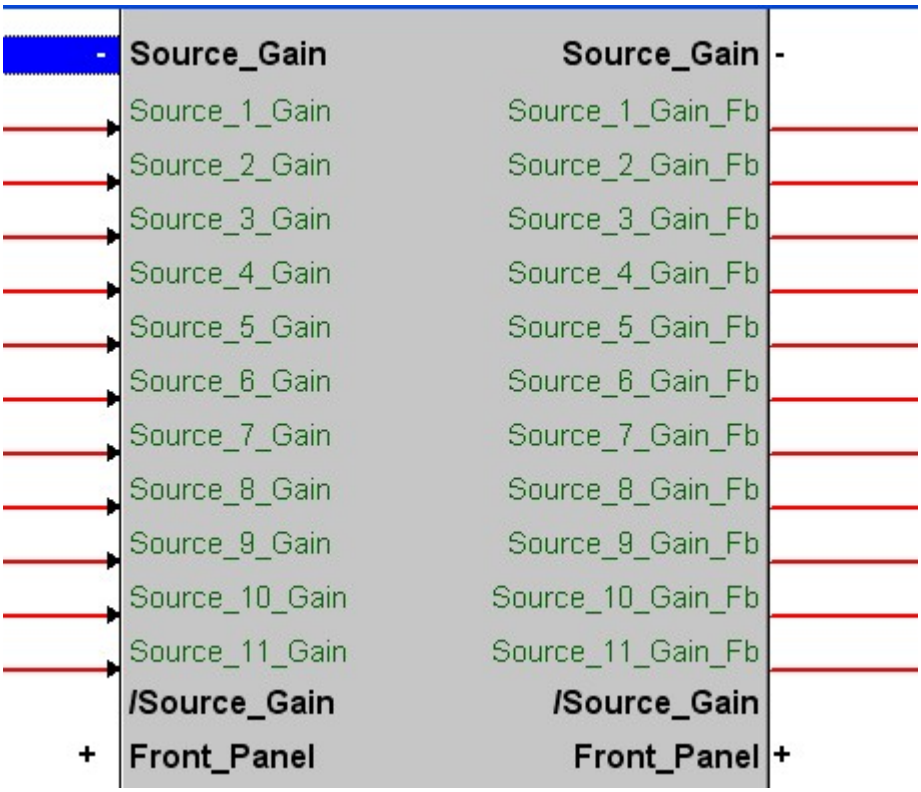
Advanced Usage

Use the [~BeginGroup~] and [~EndGroup~] specifiers to group together similar signals. The user of the module can then expand or collapse the group name to view the signals in the group.

The [~BeginGroup~] specifier must be followed by a group name, e.g., "[~BeginGroup~]Source_Gain".

The group name will be shown on the module in bold letters with a plus sign, signifying that the list can be expanded.

When expanded, the list will show the signals defined within the group.



The end of the group must be indicated by the [~EndGroup~] specifier. The "end group" specifier can be followed by the group name, although this is optional. As shown in the above example, the end of the group will be shown on the module as the group name (in bold letters) preceded by a forward slash.

There are various ways to create a group. For example, you can manually type the [~BeginGroup~] *group_name* and [~EndGroup~] specifiers and define the signals between them. Or you can use the context menu as follows:

To create a group

1. Right-click a blank signal and point to **Create Input/Output Group**.

2. Select the desired option from the context menu. For example, click **Insert Empty Group** (or press Alt+G). This will automatically set up the "begin group" and "end group" specifiers.

3. Type the group name in the text field and click **OK**.

4. Define the signals in the group. The first signal in the group will be positioned under the group name.

You can also use the context menu options to insert the "begin group" and "end group" specifiers separately, for cases where signals may already exist and you want to go back and group them. To do this, right-click the signal that's positioned *before* the first signal in the group, point to **Create Input/Output Group** and select **Insert Group Begin**. Then select the last signal in the group and choose the **Insert Group End** option.

Groups cannot be nested; that is, you cannot create a group inside of another group.

[~UNUSED~]

You can create a visual space between signal cues by using the special signal name [~UNUSED~].

Although the UNUSED qualifier results in a blank space between signals in the module's symbol, there is technically still a signal associated with the gap. (SIMPL forces UNUSED to a signal type to prevent compiler errors).

SIMPL will delete any signal that's attached to an UNUSED signal if a change to the Argument Definition causes a previously defined signal to shift into the UNUSED position.

For example, if you define the signal cues as follows:

i1 = vcr_play

i2 = vcr_stop

i3 = [~UNUSED~]

i4 = vcr_power

If you drop this module into your program, you'll see "vcr_play", "vcr_stop", a gap, and then "vcr_power".

If you attach 3 signals to the module in a SIMPL program, and then you subsequently edit the module as follows:

i1 = vcr_play

i2 = vcr_stop

i3 = vcr_pause

i4 = [~UNUSED~]

i5 = vcr_power

If you now use this module, you'll get a warning that the signal that was tied to the 4th position - which was vcr_power, but is now inaccessible because it was shifted to an UNUSED. SIMPL will delete whatever signal you had tied there.

Optional signals

You can enclose a signal cue in brackets [ ] to make the signal optional.

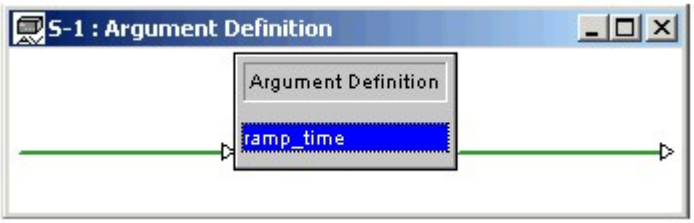
When designating a signal as optional you must ensure that you associate the signal cue with a signal that is optional on a logic symbol.

For example, if you define an optional signal on the Argument Definition, called [out], and you connect this to the output of a Toggle symbol, this is not legal and may cause the program not to work. In addition, no compiler error will be generated as the result of incorrectly connecting the signal to a non-optional input/output.

Define Parameters

To define a parameter

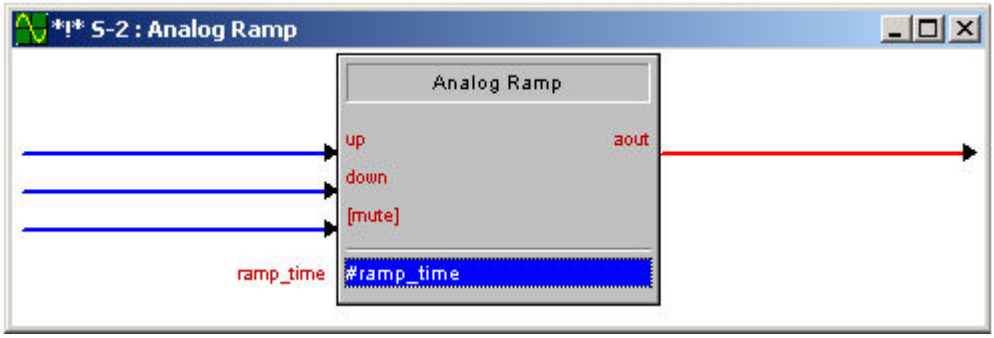
1. Name the parameter field in the Argument Definition.



2. Create a reference to the parameter field by defining a parameter in the associated logic symbol; prefix the parameter name with the pound character (no space between the pound character and the first letter of the parameter name).

   The pound character signifies that the parameter name is a placeholder for the actual parameter value the user will enter when they use the module in a program. (It is not recommended to use the pound character itself as part of the parameter name, as this will lead to unexpected results.)

   The parameter name is case-sensitive; if the case does not match, the parameter substitution will not work.



To set a default value for a parameter

- Use the == operator, which means that the value that follows is the default value. When the module is added to a program, the parameter will be set to the default value until the user sets a different value.

  For example, to set a default value of 5s for a parameter named ramp_time:

a) The parameter in the Argument Definition will be: ramp_time==5s.

b) The associated parameter in the logic symbol will be: #ramp_time==5s.

-or-

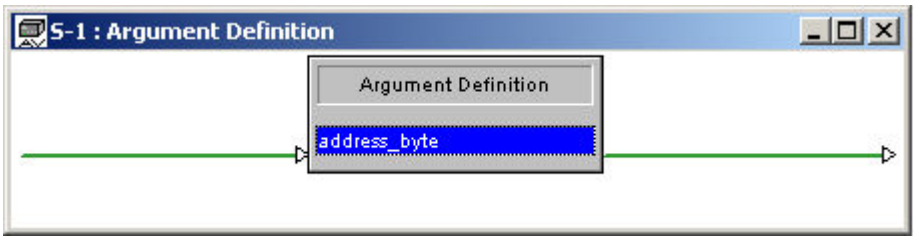
1. Right-click the parameter and select **Edit Parameter Properties**.
2. Select the **Default Value** check box.
3. Enter the default value in the text field.

To set a partial parameter substitution

SIMPL allows you to define a parameter that consists of fixed and variable portions; this is called partial parameter substitution.

For example, suppose you want to insert a Device ID into the middle of a parameter that is formatted as a string:

1. Name the parameter in the Argument Definition.

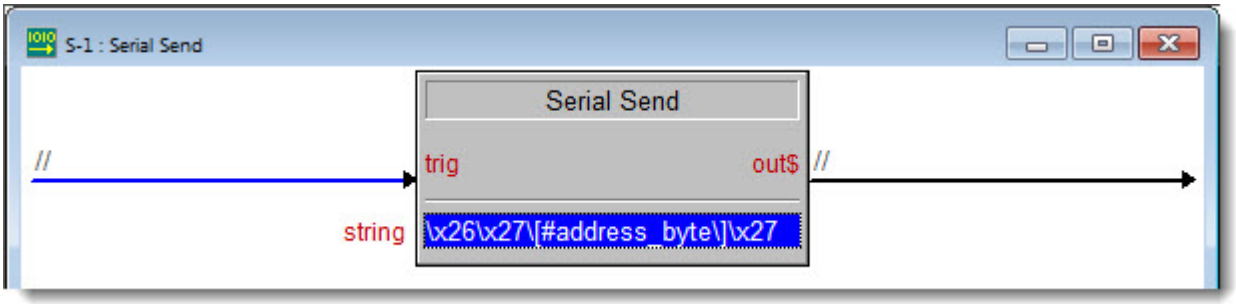


Connect the parameter to the logic symbol in the module. Precede the variable portion of the parameter with **\[#** and follow it with **\]**.

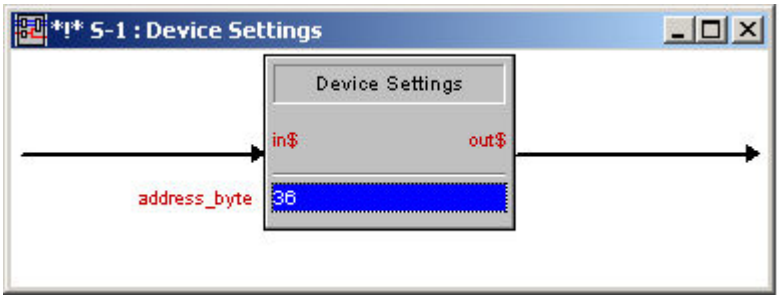
In the above example the **\[#** and **\]** are escape codes used to denote the start and end of the partial replacement. This signifies that the portion between **\[#** to **\]** will be replaced by the parameter value the user will enter when they use the module in a program.

The parameter name is case-sensitive; if the case does not match the partial substitution will not work.

The pound symbol "#" must appear before the parameter name (e.g., #address_byte) in the string.



The completed module will have a parameter called "address_byte". The fixed portion of the string will not be visible to the user of the module.



In the example above, the serial string that is transmitted in the final program output will be:

\x25\x26\x36\x27

This technique can be used anywhere; it is not restricted to a string.

For example, in an Analog Initialize:

00\[#address_byte\]h

Given a value of 36, the Analog Initialize output would result in 0036h after compilation.

Signal/Parameter Properties

Digital Input Properties

Digital Output Properties

Analog Input Properties

Analog Output Properties

Serial Input Properties

Serial Output Properties

Replace // with 0

The module header provides the option to replace all instances of the special signal name // with 0s (zeros).

To select this option, click the **OK to replace // signal with 0...** check box on the General Information tab.

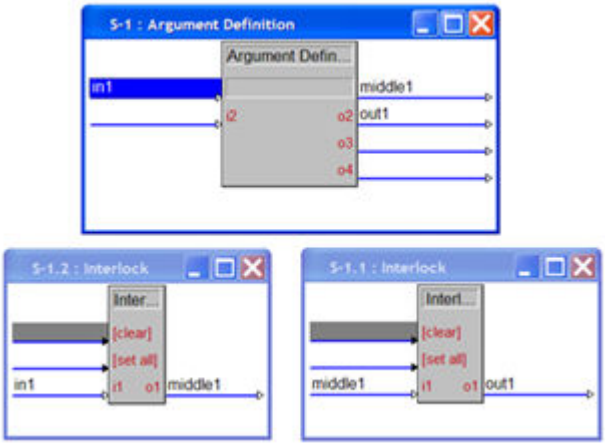
The signal name // is normally assigned when an input or output of the module will not be used.

By default, the compiler will convert each instance of // to a reserved signal name not being used elsewhere in the program. Because each instance of // is converted individually, none of the // signals are connected together and thus present no jamming issues. However, the use of reserved signal names throughout a program can result in using more memory than necessary.

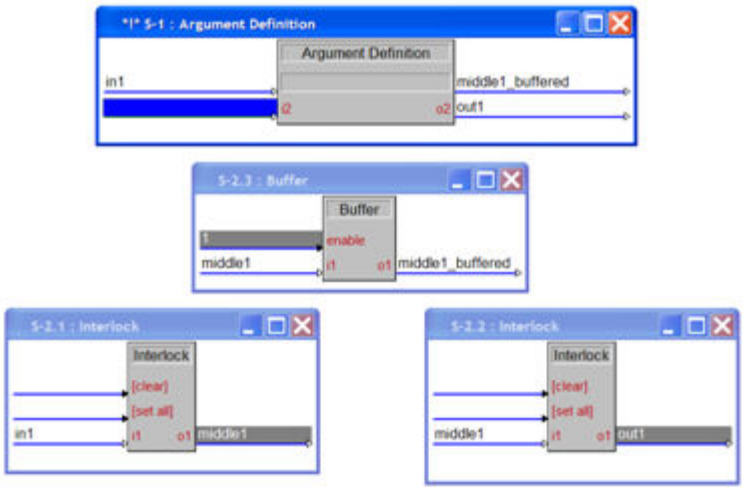
When the "OK to replace // signal with 0" option is selected, the compiler will replace instances of // with 0s. This removes the reserved signals from the program and saves memory.

If good programming practices are not followed, placing 0s on signals can result in errors such as jamming or breaking of the connections between logic symbols in the module.

For example, if a signal is connecting two Interlocks inside the module (<middle1> in the example below), and that signal is also defined on the Argument Definition, then putting a 0 on the signal at compile time will also put a 0 on the input and output of the two Interlocks, thereby breaking the connection between those two symbols.

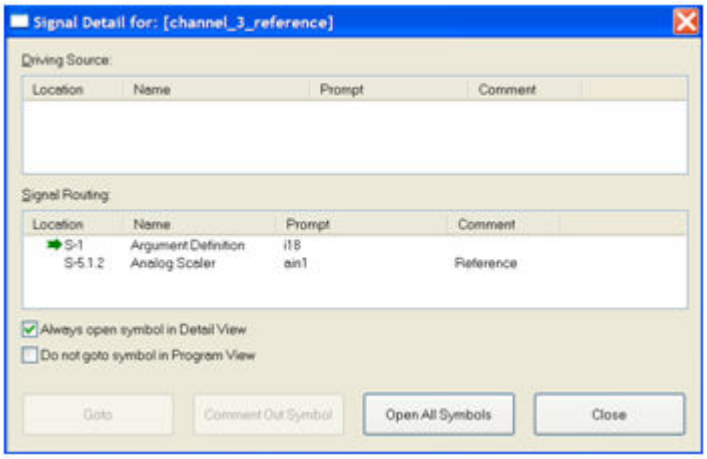


Good programming practice is to buffer the output of the Argument Definition (<middle1_buffered> in the example below). In this way a 0 can be placed on the output without affecting functionality.

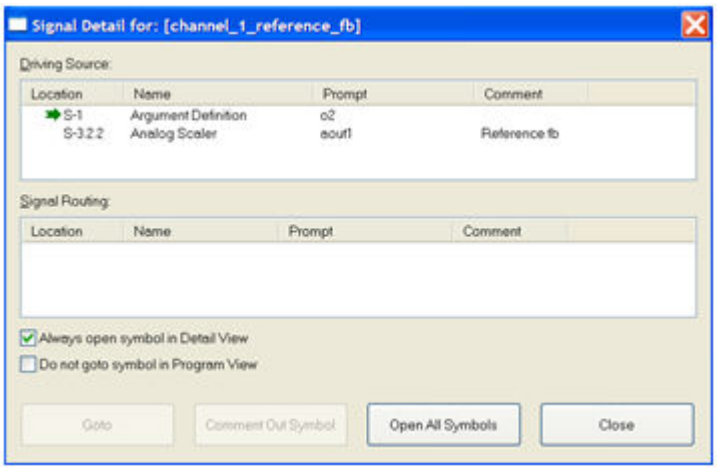


The "OK to replace // signal with 0" option should <u>only</u> be checked if the module conforms to the following rules:

1. Each input to the module's Argument Definition must only show Signal Routing to destinations, and have no Driving Sources, as shown in the Signal Detail dialog. To open the dialog, select the signal and press F2.



2. Each output of the Argument Definition must only show Driving Sources, with no Signal Routing to destinations, as shown in the Signal Detail dialog.



3. If the module contains another module that is connected to an input or output of the Argument Definition, each signal in the nested module will also need to be evaluated as described above.

Add User Interface Connection Logic to a SIMPL Program

Upon creating a user interface connection logic module and saving it in the designated user module directory, the module can be connected to any interface in a SIMPL program. Connection logic modules are identified by a red connection logic icon.

To view which devices the module is designed to control, right-click the module in the Symbol Library and point to **View devices supported for import of logic**.

To add user interface connection logic

1. In the Program View tree, expand the slot where the touchpanel or keypad is located so that the interface is visible in the tree.

2. Drag the user interface connection logic module from the User Modules directory to the Logic folder in the Program View tree.

3. Right-click the module in the Logic folder and drop it on the interface.

4. Click **Import User Interface Connection Logic**.

5. On the "Import User Interface Logic" dialog, specify optional offsets from the programmed input and output positions on the module. For example, if the digital inputs of the module begin at <**press1**> and an offset of 5 is specified in the Digital Inputs field, the corresponding digital outputs of the touchpanel symbol will begin at <**press5**>.

   Type the desired offsets in the text fields, or keep the default value of 1 for no offset. Keep in mind that any offset values typed in the Inputs field will be mapped to outputs on the touchpanel

symbol detail. Similarly, offset values typed in the Outputs field will be mapped to inputs on the touchpanel symbol detail.

If you specify an offset in an Inputs field, this will automatically set the Outputs field to the same offset value. However, you can always set the Outputs field to a different value.

6. Still on the "Import User Interface Logic" dialog, type an optional prefix for all connection logic signals that will be defined in the touchpanel, or leave the Signal Naming field blank for no prefix.

7. Click **OK** to close the "Import User Interface Logic" dialog. The module will now be automatically connected to the interface.

Add Crestron Device Connection Logic to a SIMPL Program

Upon creating a user Crestron device connection logic module and saving it in the designated user module directory, the module can be connected to the compatible Crestron device in the SIMPL program. Connection logic modules are identified by a red connection logic icon.

To view the device the module is designed to control, right-click the module in the Symbol Library and point to **View devices supported for import of logic**.

To add Crestron device connection logic

1. In the Program View tree, expand the slot where the Crestron device is located so that the device is visible in the tree.

2. Drag the Crestron device connection logic module from the User Modules directory to the Logic folder in the Program View tree.

3. Right-click the module and drop it on the Crestron device.

4. Click **Import Device Logic**.

5. Type an optional prefix for signals that will be imported into the device, or leave the Signal Naming field blank for no prefix.

Add Front Panel Connection Logic to a SIMPL Program

Upon creating a user front panel project connection logic module and saving it in the designated user module directory, the module can be connected to the (compatible) front panel slot in the SIMPL

program. Connection logic modules are identified by a red connection logic icon.

To view which devices the module is designed to control, right-click the module in the Symbol Library and point to **View devices supported for import of logic**.

To add front panel connection logic

1. Drag the front panel connection logic module from the User Modules directory to the Logic folder in the Program View tree.
2. Right-click the module in the Logic folder and drop it on the Front Panel slot of the control system.
3. Click **Import Front Panel**.
4. Upon importing the front panel, click **Edit Front Panel** on the **Project** menu. This will open the Front Panel Editor.
5. Click **Synchronize Signal Names** on the **Panel** menu (of the Front Panel Editor).
6. Close the Front Panel Editor. The module will now be connected to the front panel.

Edit an Existing Module

You can use an existing user or Crestron module as the starting point for your own user modules.

You can also modify a user module.

To use an existing module as a starting point for creating a new module

1. Locate the User or Crestron module in the Symbol Library.
2. Right-click the module and point to **Edit/Import Module**.
   - To open the module in the current instance of SIMPL (thereby closing the current program), click **Open in this Instance**.
   - To open the module in a new instance of SIMPL, click **Open in a New Instance**.
3. Open the module header, review the target control systems selection and if necessary make changes as desired

   IMPORTANT:

   - SIMPL module(s): The new target control system selection will take place ONLY after the module is saved

- SIMPL+ module(s): The new target control system selection will take place ONLY after the module is recompiled

4. Edit the module as desired.

5. Use the **Save As** command to save the module under a new name. You must save the module in your designated user module directory.

To edit a user module

1. Open the module as described above.

2. Open the module header, review the target control systems selection and if necessary make changes as desired

   IMPORTANT:

   - SIMPL module(s): Even though 4-series may appear selected, the new target control system selection will take place ONLY after the module is saved

   - SIMPL+ module(s): Even though 4-series button may appear selected, the new target control system selection will take place ONLY after the module is recompiled

3. Edit the module as desired.

4. Use the **Save** command to save the module. You must save the module in your designated user module directory.

More:

SIMPL/SIMPL+ Modules Targeting 4-Series Control Systems

Updating an Existing Module's I/O

Modules frequently will require the Inputs/Outputs/Parameters (a.k.a I/O/P) to be updated. Best Practices recommend retaining the I/O/P order, type and cues--if the module has ever been published/used in a program.

If an already published module needs to be updated, the following guidelines are recommended:

1. If the module is getting a new name, you can change the Input/Output/Parameter order and will not have to worry about resync issues resulting from inserting/removing I/O in the middle of the lists.

a. Keep the existing cues the same name

b. SIMPL does not support replacing modules but, when the new module is available, the programmer can add the new module and copy the programming from the old module to the new module. With that said:

   i. For I/O being removed, you can change those cues to an [~Unused2~] label which will hide that cue in the new module and not leave an open gap in the I/O Lists.

      1. [~Unused~]= skip space for GUI and take no join

      2. [~Unused2~] = no visual gap/join number gap

      3. [~Unused3~] = visual gap & join gap

c. SIMPL, when copying the programming, will look for cue matches to move the programming over.

2. If the module is keeping the exact same name, you will need to worry about resync issues, especially if the current I/O order or type is changed, you can lose programming:

   a. Keep all all existing I/O in the same order.

   b. Keep all existing cues the same (to aid the programmer in moving programming over that could not be copied by SIMPL).

   c. For I/O being removed, you can change those cues to an [~Unused2~] label which will hide that cue in the new module and not leave a visual gap in the I/O Lists.

      i. [~Unused~] = skip space for GUI and take no join

      ii. [~Unused2~] = no visual gap/join number gap

      iii. [~Unused3~] = visual gap & join gap

   d. Add new I/O to the bottom of the list.

   e. When the program is opened and the new module is in the Symbol Library

      i. SIMPL will tell the programmer they need to resync the program.

      ii. Because the module name is the same, SIMPL will attempt to assign existing programming based on I/O position.

      iii. For the I/O that were converted to [~Unused~]/[~Unused2~]/[~Unused3~], SIMPL will remove the signals on those items and notify the user of such.